🗄️ COMANDOS SQL SERVER, POSTGRESQL E MYSQL - GUIA COMPLETO

🔌 CONEXÃO E CONFIGURAÇÃO

SQL Server

bash

```
# Conectar via sqlcmd
sqlcmd -S localhost -U sa -P sua_senha
sqlcmd -S servidor.database.windows.net -U usuario -P senha -d banco_de_dados

# Conectar via PowerShell
Connect-DbaInstance -SqlInstance localhost -SqlCredential (Get-Credential)

# Conectar via Azure Data Studio
```

```
# Interface gráfica - download Microsoft Azure Data Studio
```

PostgreSQL

bash

```
# Conectar via psql
psql -h localhost -U postgres -d banco_de_dados
psql postgresql://usuario:senha@localhost:5432/banco_de_dados

# Conectar com variáveis de ambiente
export PGHOST=localhost PGPORT=5432 PGDATABASE=meubanco PGUSER=postgres
PGPASSWORD=senha
psql
```

```
# Conectar via pgAdmin (interface gráfica)
```

MySQL

bash

```
# Conectar via mysql client
mysql -h localhost -u root -p
mysql -u usuario -p -h localhost -P 3306 banco_de_dados

# Conectar com variáveis de ambiente
export MYSQL_HOST=localhost MYSQL_USER=root MYSQL_PASSWORD=senha
MYSQL_DATABASE=meubanco
mysql
```

```
# Conectar via MySQL Workbench (interface gráfica)
```

📊 COMANDOS BÁSICOS DE GERENCIAMENTO

Bancos de Dados

sql

```
-- SQL Server
CREATE DATABASE meubanco;
USE meubanco;
DROP DATABASE meubanco;
SELECT name FROM sys.databases;

-- PostgreSQL
CREATE DATABASE meubanco;
\c meubanco;  -- Conectar ao banco
DROP DATABASE meubanco;
\l  -- Listar bancos

-- MySQL
CREATE DATABASE meubanco;
USE meubanco;
DROP DATABASE meubanco;
```

```
SHOW DATABASES;
```

Tabelas

sql

```sql
-- SQL Server
CREATE TABLE usuarios (
    id INT IDENTITY(1,1) PRIMARY KEY,
    nome NVARCHAR(100) NOT NULL,
    email NVARCHAR(255) UNIQUE,
    data_criacao DATETIME2 DEFAULT GETDATE()
);

SELECT * FROM sys.tables;
EXEC sp_help 'usuarios';  -- Informações da tabela

-- PostgreSQL
CREATE TABLE usuarios (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE,
    data_criacao TIMESTAMP DEFAULT NOW()
);

\dt  -- Listar tabelas
\d usuarios  -- Descrever tabela

-- MySQL
CREATE TABLE usuarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE,
    data_criacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

SHOW TABLES;
```

```sql
DESCRIBE usuarios;
```

🔍 CONSULTAS SQL - CRUD

SELECT

sql

```sql
-- Todos os bancos
SELECT * FROM tabela;
SELECT coluna1, coluna2 FROM tabela;
SELECT DISTINCT coluna FROM tabela;

-- WHERE
SELECT * FROM usuarios WHERE idade > 18;
SELECT * FROM produtos WHERE preco BETWEEN 10 AND 100;
SELECT * FROM clientes WHERE nome LIKE 'João%';

-- ORDER BY
SELECT * FROM produtos ORDER BY preco DESC;
SELECT * FROM usuarios ORDER BY nome ASC, data_cadastro DESC;

-- LIMIT/OFFSET
-- SQL Server
SELECT TOP 10 * FROM usuarios ORDER BY id DESC;

-- PostgreSQL
SELECT * FROM usuarios ORDER BY id DESC LIMIT 10 OFFSET 20;

-- MySQL
```

```sql
SELECT * FROM usuarios ORDER BY id DESC LIMIT 10 OFFSET 20;
```

INSERT, UPDATE, DELETE

```sql
-- INSERT
INSERT INTO usuarios (nome, email) VALUES ('João', 'joao@email.com');
INSERT INTO produtos (nome, preco) VALUES
    ('Produto A', 10.50),
    ('Produto B', 20.00);

-- UPDATE
UPDATE usuarios SET nome = 'João Silva' WHERE id = 1;
UPDATE produtos SET preco = preco * 1.1;  -- Aumenta 10%

-- DELETE
DELETE FROM usuarios WHERE id = 1;
```

```sql
DELETE FROM logs WHERE data < '2023-01-01';
```

JOINs

```sql
-- INNER JOIN
SELECT u.nome, p.descricao
FROM usuarios u
INNER JOIN pedidos p ON u.id = p.usuario_id;

-- LEFT JOIN
SELECT u.nome, p.descricao
FROM usuarios u
LEFT JOIN pedidos p ON u.id = p.usuario_id;

-- RIGHT JOIN (PostgreSQL e MySQL)
SELECT u.nome, p.descricao
FROM usuarios u
RIGHT JOIN pedidos p ON u.id = p.usuario_id;

-- FULL OUTER JOIN (SQL Server e PostgreSQL)
SELECT u.nome, p.descricao
FROM usuarios u
FULL OUTER JOIN pedidos p ON u.id = p.usuario_id;
```

🛠️ COMANDOS AVANÇADOS

Funções de Agregação

```sql
-- COUNT, SUM, AVG, MAX, MIN
SELECT COUNT(*) FROM usuarios;
SELECT AVG(preco) FROM produtos;
SELECT MAX(data_cadastro) FROM usuarios;
SELECT categoria, COUNT(*) FROM produtos GROUP BY categoria;

-- HAVING
SELECT categoria, AVG(preco) as media_preco
FROM produtos
GROUP BY categoria
HAVING AVG(preco) > 50;
```

Subconsultas

sql

```sql
-- Subconsulta WHERE
SELECT nome FROM produtos
WHERE preco > (SELECT AVG(preco) FROM produtos);

-- Subconsulta FROM
SELECT categoria, media_preco
FROM (
    SELECT categoria, AVG(preco) as media_preco
    FROM produtos
    GROUP BY categoria
) subquery
WHERE media_preco > 100;

-- EXISTS
SELECT nome FROM clientes c
WHERE EXISTS (
    SELECT 1 FROM pedidos p
    WHERE p.cliente_id = c.id AND p.valor > 1000
);
```

Views

sql

```sql
-- SQL Server
CREATE VIEW vw_usuarios_ativos AS
SELECT id, nome, email
FROM usuarios
WHERE ativo = 1;

SELECT * FROM vw_usuarios_ativos;

-- PostgreSQL
CREATE OR REPLACE VIEW vw_usuarios_ativos AS
SELECT id, nome, email
FROM usuarios
WHERE ativo = true;

-- MySQL
```

```sql
CREATE VIEW vw_usuarios_ativos AS
SELECT id, nome, email
FROM usuarios
WHERE ativo = 1;
```

🔐 SEGURANÇA E USUÁRIOS

Usuários e Permissões

```sql
-- SQL Server
CREATE LOGIN novo_usuario WITH PASSWORD = 'senha123';
CREATE USER novo_usuario FOR LOGIN novo_usuario;
GRANT SELECT, INSERT ON usuarios TO novo_usuario;
DENY DELETE ON usuarios TO novo_usuario;

-- PostgreSQL
CREATE USER novo_usuario WITH PASSWORD 'senha123';
GRANT CONNECT ON DATABASE meubanco TO novo_usuario;
GRANT SELECT, INSERT ON usuarios TO novo_usuario;
GRANT USAGE ON SCHEMA public TO novo_usuario;

-- MySQL
CREATE USER 'novo_usuario'@'localhost' IDENTIFIED BY 'senha123';
GRANT SELECT, INSERT ON meubanco.* TO 'novo_usuario'@'localhost';
FLUSH PRIVILEGES;
```

Backup e Restore

```bash
# SQL Server
# Backup
sqlcmd -S localhost -U sa -P senha -Q "BACKUP DATABASE meubanco TO
DISK='/backup/meubanco.bak'"

# Restore
```

```
sqlcmd -S localhost -U sa -P senha -Q "RESTORE DATABASE meubanco FROM
DISK='/backup/meubanco.bak'"

# PostgreSQL
# Backup
pg_dump -h localhost -U postgres meubanco > backup.sql
pg_dump -h localhost -U postgres -Fc meubanco > backup.dump  # Formato customizado

# Restore
psql -h localhost -U postgres -d meubanco < backup.sql
pg_restore -h localhost -U postgres -d meubanco backup.dump

# MySQL
# Backup
mysqldump -h localhost -u root -p meubanco > backup.sql
mysqldump -h localhost -u root -p --databases meubanco outrobanco > backup.sql

# Restore
```

```
mysql -h localhost -u root -p meubanco < backup.sql
```

⚡ PERFORMANCE E OTIMIZAÇÃO

Índices

```
sql
```

```
-- SQL Server
CREATE INDEX idx_usuarios_email ON usuarios(email);
CREATE UNIQUE INDEX idx_usuarios_cpf ON usuarios(cpf);
DROP INDEX idx_usuarios_email ON usuarios;

-- PostgreSQL
CREATE INDEX idx_usuarios_email ON usuarios(email);
CREATE INDEX CONCURRENTLY idx_usuarios_nome ON usuarios(nome);  -- Não bloqueia
tabela
DROP INDEX idx_usuarios_email;

-- MySQL
CREATE INDEX idx_usuarios_email ON usuarios(email);
CREATE UNIQUE INDEX idx_usuarios_cpf ON usuarios(cpf);
```

```
DROP INDEX idx_usuarios_email ON usuarios;
```

Explain Plans

sql

```sql
-- SQL Server
SET SHOWPLAN_ALL ON;
SELECT * FROM usuarios WHERE email = 'teste@email.com';

-- PostgreSQL
EXPLAIN ANALYZE SELECT * FROM usuarios WHERE email = 'teste@email.com';

-- MySQL
EXPLAIN SELECT * FROM usuarios WHERE email = 'teste@email.com';
```

Estatísticas e Monitoramento

sql

```sql
-- SQL Server
SELECT * FROM sys.dm_db_index_usage_stats;
DBCC SHOW_STATISTICS ('usuarios', 'idx_usuarios_email');

-- PostgreSQL
SELECT schemaname, tablename, seq_scan, seq_tup_read, idx_scan, idx_tup_fetch
FROM pg_stat_user_tables;

ANALYZE usuarios;  -- Atualizar estatísticas

-- MySQL
SHOW INDEX FROM usuarios;
ANALYZE TABLE usuarios;  -- Atualizar estatísticas
```

🔄 TRANSACTIONS E CONCORRÊNCIA

Transactions

```sql
-- SQL Server
BEGIN TRANSACTION;
    UPDATE conta SET saldo = saldo - 100 WHERE id = 1;
    UPDATE conta SET saldo = saldo + 100 WHERE id = 2;
COMMIT TRANSACTION;

-- Rollback em caso de erro
BEGIN TRY
    BEGIN TRANSACTION;
        -- operações
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;
    THROW;
END CATCH

-- PostgreSQL
BEGIN;
    UPDATE conta SET saldo = saldo - 100 WHERE id = 1;
    UPDATE conta SET saldo = saldo + 100 WHERE id = 2;
COMMIT;

-- MySQL
START TRANSACTION;
    UPDATE conta SET saldo = saldo - 100 WHERE id = 1;
    UPDATE conta SET saldo = saldo + 100 WHERE id = 2;
    COMMIT;
```

Isolation Levels

```sql
-- SQL Server
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- PostgreSQL
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

```sql
-- MySQL
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

📊 TIPOS DE DADOS ESPECÍFICOS

Tipos de Dados por Banco

```sql
-- SQL Server
CREATE TABLE exemplo_tipos (
    id INT IDENTITY PRIMARY KEY,
    nome NVARCHAR(100),
    preco DECIMAL(10,2),
    data_criacao DATETIME2,
    ativo BIT,
    json_data NVARCHAR(MAX),  -- JSON
    geo_data GEOGRAPHY,       -- Dados geográficos
    xml_data XML              -- Dados XML
);

-- PostgreSQL
CREATE TABLE exemplo_tipos (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100),
    preco NUMERIC(10,2),
    data_criacao TIMESTAMP,
    ativo BOOLEAN,
    json_data JSONB,          -- JSON binário (eficiente)
    array_data TEXT[],        -- Array
    geo_data POINT            -- Dados geográficos
);

-- MySQL
CREATE TABLE exemplo_tipos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100),
    preco DECIMAL(10,2),
    data_criacao DATETIME,
    ativo TINYINT(1),
    json_data JSON,           -- Dados JSON
    enum_data ENUM('ativo', 'inativo')  -- Enumeração
```

```
);
```

🐳 DOCKER PARA BANCOS DE DADOS

Docker Commands

```bash
# SQL Server
docker run -e "ACCEPT_EULA=Y" -e "SA_PASSWORD=Senha123!" \
  -p 1433:1433 --name sqlserver \
  -d mcr.microsoft.com/mssql/server:2019-latest

# PostgreSQL
docker run --name postgres \
  -e POSTGRES_PASSWORD=senha123 \
  -e POSTGRES_DB=meubanco \
  -p 5432:5432 \
  -d postgres:15

# MySQL
docker run --name mysql \
  -e MYSQL_ROOT_PASSWORD=senha123 \
  -e MYSQL_DATABASE=meubanco \
  -p 3306:3306 \
  -d mysql:8.0
```

Docker Compose

```yaml
version: '3.8'

services:
  sqlserver:
    image: mcr.microsoft.com/mssql/server:2019-latest
    environment:
      SA_PASSWORD: "Senha123!"
      ACCEPT_EULA: "Y"
```

```
    ports:
      - "1433:1433"
    volumes:
      - sqlserver_data:/var/opt/mssql

  postgres:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: senha123
      POSTGRES_DB: meubanco
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: senha123
      MYSQL_DATABASE: meubanco
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql

volumes:
  sqlserver_data:
  postgres_data:
```

```
  mysql_data:
```

🔧 SCRIPTS ÚTEIS

Script de Monitoramento

sql

```sql
-- SQL Server - Informações do Banco
SELECT
    name as database_name,
    state_desc,
    recovery_model_desc,
    log_reuse_wait_desc
FROM sys.databases;
```

```sql
-- PostgreSQL - Conexões Ativas
SELECT
    datname as database_name,
    usename as username,
    client_addr as client_address,
    state
FROM pg_stat_activity;

-- MySQL - Status do Servidor
SHOW STATUS LIKE 'Threads_connected';
```

```
SHOW PROCESSLIST;
```

Script de Manutenção

```
sql
```

```sql
-- SQL Server - Rebuild Indexes
EXEC sp_MSforeachtable 'ALTER INDEX ALL ON ? REBUILD';

-- PostgreSQL - Vacuum (limpeza)
VACUUM ANALYZE;

-- MySQL - Otimização de Tabelas
```

```
OPTIMIZE TABLE usuarios, produtos, pedidos;
```

Script de Backup Automático

```
bash
```

```bash
#!/bin/bash
# backup-databases.sh

DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="/backup/$DATE"

mkdir -p $BACKUP_DIR

# Backup SQL Server
/opt/mssql-tools/bin/sqlcmd -S localhost -U sa -P $SA_PASSWORD -Q "BACKUP DATABASE
[meubanco] TO DISK='$BACKUP_DIR/sqlserver.bak'"
```

```
# Backup PostgreSQL
pg_dump -h localhost -U postgres meubanco > "$BACKUP_DIR/postgres.sql"

# Backup MySQL
mysqldump -h localhost -u root -p$MYSQL_PASSWORD meubanco >
"$BACKUP_DIR/mysql.sql"

# Compactar
tar -czf "/backup/backup_$DATE.tar.gz" "$BACKUP_DIR"

# Limpar
rm -rf "$BACKUP_DIR"
```

```
echo "Backup concluído: /backup/backup_$DATE.tar.gz"
```

💡 DICAS ESPECÍFICAS POR BANCO

SQL Server

```
sql
```

```
-- Consulta informações do sistema
SELECT @@VERSION;  -- Versão do SQL Server
SELECT DB_NAME();  -- Banco atual

-- Dynamic Management Views
SELECT * FROM sys.dm_exec_connections;  -- Conexões
SELECT * FROM sys.dm_exec_sessions;     -- Sessões

-- Temporal Tables (histórico de alterações)
CREATE TABLE produtos (
    id INT PRIMARY KEY,
    nome NVARCHAR(100),
    preco DECIMAL(10,2),
    valid_from DATETIME2 GENERATED ALWAYS AS ROW START,
    valid_to DATETIME2 GENERATED ALWAYS AS ROW END,
    PERIOD FOR SYSTEM_TIME (valid_from, valid_to)
```

```
    ) WITH (SYSTEM_VERSIONING = ON);
```

PostgreSQL

```sql
-- Extensões úteis
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";  -- UUID
CREATE EXTENSION IF NOT EXISTS "pgcrypto";   -- Criptografia

-- Consultas JSON
SELECT * FROM usuarios WHERE json_data->>'status' = 'ativo';
SELECT nome, json_data->>'email' as email FROM usuarios;

-- Full Text Search
SELECT * FROM documentos
WHERE to_tsvector('portuguese', conteudo) @@ to_tsquery('postgres & banco');

-- Window Functions
SELECT nome, salario, AVG(salario) OVER (PARTITION BY departamento) as
media_departamento
FROM funcionarios;
```

MySQL

```sql
-- Engine de armazenamento
SHOW ENGINES;
CREATE TABLE exemplo (id INT) ENGINE=InnoDB;

-- Variáveis de configuração
SHOW VARIABLES LIKE 'innodb_buffer_pool_size';
SET GLOBAL innodb_buffer_pool_size = 1073741824;  -- 1GB

-- Partitioning
CREATE TABLE vendas (
    id INT AUTO_INCREMENT,
    data_venda DATE,
    valor DECIMAL(10,2),
    PRIMARY KEY (id, data_venda)
) PARTITION BY RANGE (YEAR(data_venda)) (
    PARTITION p2022 VALUES LESS THAN (2023),
    PARTITION p2023 VALUES LESS THAN (2024)
);
```

Este guia cobre todos os comandos essenciais para trabalhar com SQL Server, PostgreSQL e MySQL! 🗄️